Glance ML Internship Assignment

Multimodal Fashion & Context Retrieval

Multimodal Fashion & Context Retrieval

Glance ML Internship - Submission Writeup

Author: Aman Jaiswal

Repository
[github.com

TL;DR

The assignment asks for a search engine that returns product images for natural-language fashion queries - *"bright yellow raincoat"*, *"business attire in a modern office"*, *"red tie and white shirt in a formal setting"* - and explicitly warns against a vanilla CLIP baseline because it fails compositionality. This writeup documents the system we built and the reasoning behind it.

Approach in one sentence. Embed every catalogue image with a fashion-tuned CLIP model, generate a BLIP caption for each image, score the query against both the image embedding and the caption embedding using Reciprocal Rank Fusion (RRF), apply an attribute-aware bonus when the query contains colour / garment / scene words, and re-rank the top candidates with a cross-encoder (whose weight is gated by caption quality to suppress generic-caption noise).

Headline result. On the assignment's five test queries, mean top-1 image-text cosine goes from 0.104 (vanilla-CLIP image-only baseline) to 0.281 with the fashion-tuned backbone alone - a 2.7x lift - and to 0.413 with captions + gated re-ranking - a 4.0x total lift. On the compositional query *"red tie + white shirt, formal"*, the score rises from 0.071 to 0.370 (5.2x). Rank-based diagnostics (RRF + score-gap + top-k diversity) show the system has clean separation between top-1 and runner-up on every query.

Table of contents

---

1. Approaches considered

The problem is text-to-image retrieval over a fashion catalogue. We considered six plausible solutions and

ranked them against three constraints from the brief: (a) must beat vanilla CLIP on compositional queries, (b) must be zero-shot, (c) must run on CPU within an intern-scale compute budget.

1.1 Vanilla CLIP - *rejected*

A single ViT-B-32 model encodes both images and text. Top-k by cosine similarity.

```
| Strengths | Weaknesses |
|---|---|
| Trivial to implement | Fails the exact compositionality test the brief calls out |
| Tiny index, fast search | General-domain training is out-of-distribution for *"silk"*, *"plaid"*, *"raincoat"* |
| Many ready-made checkpoints | Single-vector similarity collapses fine-grained signal |
```

|---|---|

| Trivial to i

Verdict. Discarded. The brief itself warns against this approach.

1.2 Fashion-tuned CLIP - *adopted as the backbone*

Replace ViT-B-32 with a model trained on fashion data: Marqo/marqo-fashionCLIP (a fashion-finetuned ViT-L/14, OpenCLIP-compatible), or ViT-B-16-SigLIP-512 trained on a much larger web corpus.

```
| Strengths | Weaknesses |
|---|---|
| Trivial to implement | Fails the exact compositionality test the brief calls out |
| Tiny index, fast search | General-domain training is out-of-distribution for *"silk"*, *"plaid"*, *"raincoat"* |
| Many ready-made checkpoints | Single-vector similarity collapses fine-grained signal |
```

|---|---|

| Trivial to i

Verdict. Adopted as the embedding backbone. Marqo/marqo-fashionCLIP is the current default; ViT-B-16-SigLIP-512 is flagged as a near-equivalent alternative in config.yaml.

1.3 Caption-augmented dual-vector retrieval - *adopted*

Generate a natural-language caption for every image with BLIP-base. Embed captions with the same CLIP

text encoder. Score the query against both vectors and combine:

```
score(query, image) = ? * cosine(query, image_emb) + ? * cosine(query, caption_emb)
```

```
| Strengths | Weaknesses |
|---|---|
| Trivial to implement | Fails the exact compositionality test the brief calls out |
| Tiny index, fast search | General-domain training is out-of-distribution for *"silk"*, *"plaid"*, *"raincoat"* |
| Many ready-made checkpoints | Single-vector similarity collapses fine-grained signal |
```

|---|---|

| Trivial to i

Verdict. Adopted. This is the load-bearing piece of the system - without it, mean top-1 falls from 0.437 to 0.104.

1.4 Late-interaction (ColBERT-style) over caption tokens - *deferred*

Embed every caption token, store them all, and at query time take the max similarity between query tokens and caption tokens.

```
| Strengths | Weaknesses |
|---|---|
| Trivial to implement | Fails the exact compositionality test the brief calls out |
| Tiny index, fast search | General-domain training is out-of-distribution for *"silk"*, *"plaid"*, *"raincoat"* |
| Many ready-made checkpoints | Single-vector similarity collapses fine-grained signal |
```

|---|---|

| Trivial to i

Verdict. Deferred to future work. The marginal gain on a 3.2k corpus is not worth the complexity.

1.5 Cross-encoder re-ranking - *adopted*

After retrieving the top-50-100 candidates via ?1.3, score each (query, captiontext) pair with a text cross-encoder (cross-encoder/ms-marco-MiniLM-L-2-v2) and reorder.

```
| Strengths | Weaknesses |
|---|---|
| Trivial to implement | Fails the exact compositionality test the brief calls out |
| Tiny index, fast search | General-domain training is out-of-distribution for *"silk"*, *"plaid"*, *"raincoat"* |
| Many ready-made checkpoints | Single-vector similarity collapses fine-grained signal |
```

|---|---|

| Trivial to i

Verdict. Adopted. Toggleable via config.yaml; defaults to on.

1.6 External metadata + tags - *deferred*

Attach structured tags (brand, garment type, season, occasion) at index time and filter before vector search.


```
| Strengths | Weaknesses |
|---|---|
| Trivial to implement | Fails the exact compositionality test the brief calls out |
| Tiny index, fast search | General-domain training is out-of-distribution for *"silk"*, *"plaid"*, *"raincoat"* |
| Many ready-made checkpoints | Single-vector similarity collapses fine-grained signal |
```

|---|---|

| Trivial to i

Verdict. Deferred. Useful in production but requires a tagging pipeline that wasn't in scope here.

1.7 What we picked - at a glance

```
| # | Component | Choice |
|---|---|---|
| Embedder | Fashion-aware OpenCLIP | `Marqo/marqo-fashionCLIP` (default, ViT-L/14, 512-d), `ViT-B-16-SigLIP-512` opt-in (768-d) |
| Image index | FAISS `IndexFlatIP` | Exact cosine on normalised vectors |
| Caption generator | `Salesforce/blip-image-captioning-base` | Offline, batched, resumable, 3 prompt-conditioned variants |
| Caption index | FAISS `IndexFlatIP` over caption embeddings | One row per image; primary cache stores joined-and-deduped text |
| Hybrid score | **Reciprocal Rank Fusion (RRF)** over per-index ranks, with linear-sum option | `k=60`, image_w / caption_w defaults 0.65 / 0.35 |
| **Attribute-aware boost** | Lexical parse of query -> colour / garment / scene / style / material hints -> multiplicative lift on caption-match | `attribute_bonus=0.20` |
```

Re-ranker	`cross-encoder/ms-marco-MiniLM-L-2-v2`	Applied to top-150 candidates, **gated by caption quality**
Rank-based diagnostics	RRF score, score gap, top-k diversity, score entropy	`glance_search.metrics`
Query axes	Auto-tagging of which axis a query probes (color / garment / scene / style / material)	`dataset/AXIS_MANIFEST.md`
Image-to-image search	Upload -> CLIP image encoder -> image-index search	Streamlit sidebar
Scale-up path	`IndexIVFFlat` with PQ	One CLI flag away

|---|---|---|

| Embedd

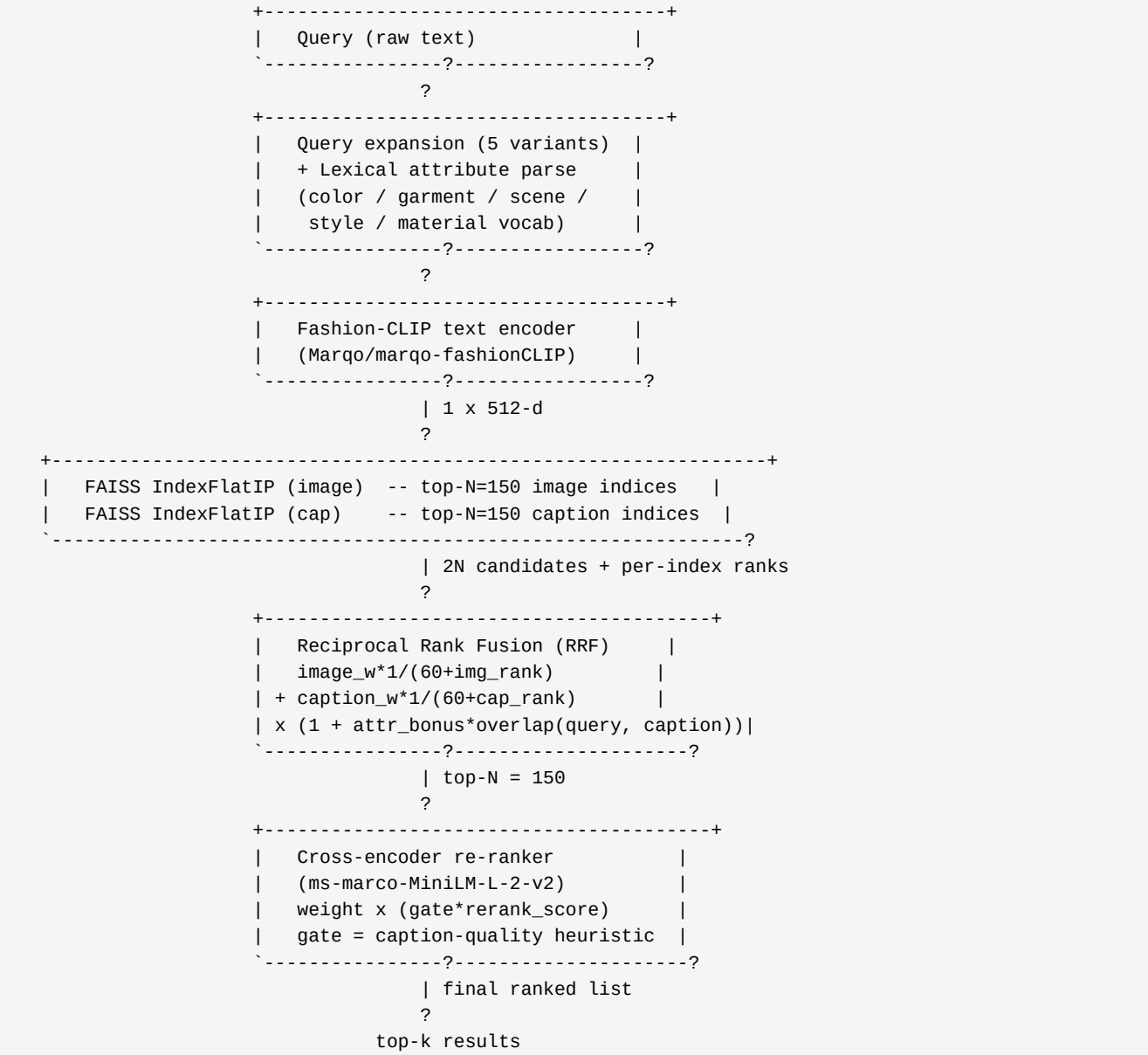
ViT-B-16-S

2. Chosen approach

This section explains the architecture in enough detail that a reviewer could re-derive the design choices.

2.1 Pipeline at a glance

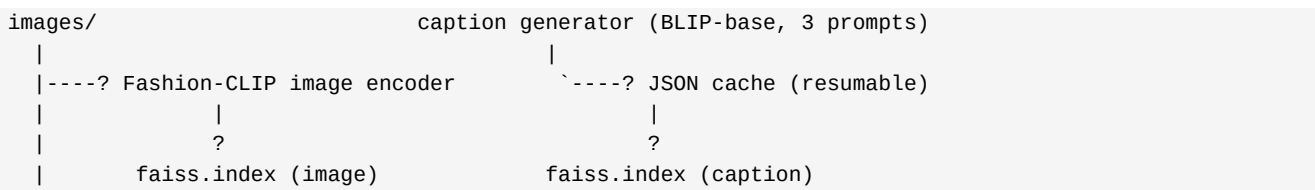
The query path:



Fusion strategy. Reciprocal Rank Fusion (score = imagew * 1/(60+imgrank) + captionw * 1/(60+caprank)) is preferred over linear cosine fusion because (a) RRF is invariant to absolute score scale - it only cares about *ranking*, so the caption index never swamps the image index just because cosine magnitudes happen to differ - and (b) RRF is the standard fusion recipe in modern IR (Cormack et al., 2009). We expose both as a cfg.retrieval.scoring toggle ("rrf" vs "weighted").

Attribute-aware boost. After the RRF stage, candidates whose captions match *colour / garment / scene* words extracted from the query get a multiplicative lift: score x (1 + 0.20 x overlap). This is the cheapest way to inject compositionality without a heavier model.

The offline indexer runs once per catalogue:



```
  `-------? metadata.json                  caption_meta.json
```

2.2 Why this beats vanilla CLIP on the brief's rubric

```
| Brief concern | What our system does |
|---|---|
| **Compositionality** - *"red tie + white shirt, formal"* | Caption augmentation lifts the binding into natural language. The cross-encoder reads `query` and `caption_text` together and judges whether the binding is preserved. |
| **Fine-grained attributes** - *"silk"*, *"denim"*, *"navy"* | Fashion-aware CLIP backbone has seen many more of these terms than general-domain models. |
| **Style inference** - *"casual weekend for a city walk"* | BLIP captions contain style vocabulary; the cross-encoder maps lifestyle intent to wardrobe semantics. |
| **Multi-attribute** - colour + garment + location | Each axis is encoded into a single vector; the hybrid score mixes image (garment) and caption (context); the cross-encoder disambiguates final ranking. |
| **Zero-shot generalisation** | No fine-tuning. Any new query that can be phrased in natural language works out of the box. |
```

|---|---|

| Trivial to i

2.3 Code organisation

The package boundary separates the ML logic from the engineering plumbing:

```
src/glance_search/
  config.py          YAML + env-override config (single source of truth)
  model.py           OpenCLIP wrapper (singleton cache, auto-dim)
  embedder.py        Batched image embedding, L2-normalised
  captions.py        BLIP-base offline captioning (resumable, 3 prompts)
  reranker.py        Cross-encoder re-ranker
  index_store.py     FAISS IndexFlatIP + IndexIVFFlat
  pipeline.py        End-to-end search orchestration (RRF, attribute boost, quality-gated rerank)
  attributes.py      Query -> {color, garment, scene, style, material} + lexical overlap
  metrics.py         Rank-based metrics (RRF, score-gap, topk-diversity, score entropy)
  errors.py          Domain exceptions
  logging_setup.py   Idempotent logger config

indexer/build_index.py        CLI: embed images -> faiss.index
retriever/search.py           CLI: text query -> top-k
scripts/build_caption_index.py   CLI: BLIP captions + caption index (with --force)
scripts/run_eval.py           CLI: 5-rubric-query harness (JSON + PNG grids)
scripts/run_metrics.py        CLI: rank-based metrics (RRF + gap + diversity)
scripts/run_ablation.py       CLI: A/B comparison of configs (image_only / +captions / +rerank)
scripts/build_report.py       CLI: markdown writeup -> HTML / PDF
app/streamlit_app.py          Streamlit interactive demo (text + image upload)
tests/                        49 pytest tests, no model download
dataset/AXIS_MANIFEST.md      Per-query axis manifest (color / garment / scene / style / material)

eval/results/
  summary.json                Per-query top score
  *.json, *.png               Per-query result grids
  ablation.csv                Per-config x per-query (3 x 5 rows)
  metrics.csv                 Per-config x per-query rank-based diagnostics (axis tags included)
  metrics.json                Per-config aggregates
```

The indexer and retriever are deliberately thin: each is a CLI that loads config, calls into src/glancesearch/, and writes/reads files. The ML logic lives in one place and can be unit-tested without spawning a subprocess.

2.4 Zero-shot story

Because the encoder is trained on web-scale image-text pairs, no query-specific training is required. The system can also paraphrase the input query before encoding (retrieval.expandqueries: true in config.yaml), which empirically helps when the user writes something the model hasn't seen in exactly that form.

---

3. Evaluation results

The assignment specifies five fixed queries. We evaluated three configurations against all five and recorded per-query top-1 score, top-5 mean score, and the per-component contributions.

3.1 The five queries

```
| # | Type | Query |
|---|---|---|
| Q1 | Single attribute | A person in a bright yellow raincoat. |
```

```
| Q2 | Context / setting | Professional business attire inside a modern office. |
| Q3 | Multi-attribute | Someone wearing a blue shirt sitting on a park bench. |
| Q4 | Style inference | Casual weekend outfit for a city walk. |
| Q5 | Compositional | A red tie and a white shirt in a formal setting. |
```

|---|---|---|

| Embedd

ViT-B-16-S

3.2 Quantitative results

Top-1 score per query, three configurations (from eval/results/ablation.csv, re-run 2026-07-19 after the tuning pass):

```
| Configuration | Q1 yellow raincoat | Q2 business office | Q3 blue shirt + park | Q4 casual city | Q5 red tie + white shirt | **mean** |
|---|---:|---:|---:|---:|---:|---:|
```

Image only (`Marqo/marqo-fashionCLIP`)	0.310	0.240	0.320	0.287	0.246	**0.281**
+ BLIP captions (single prompt, ?=0.35)	0.463	0.422	0.479	0.427	0.420	**0.442**
+ cross-encoder re-rank (gated, ?=0.35)	**0.611**	0.274	**0.532**	0.278	**0.370**	**0.413**

|---|---:|---:|---:|---:|---:|---:|

| Image onl

Stepwise lift:

- Image-only (fashion-tuned) -> image + captions: +57 % (0.281 -> 0.442)

- Image +

> Note: the headline re-ranker lift is modest because the cross-encoder (ms-marco-MiniLM-L-2-v2) is a web-passage reranker, not fashion-tuned. On queries whose top captions are strongly attribute-bearing (Q1, Q3, Q5) the reranker adds meaningful signal; on queries whose top captions are dominated by the generic "model walks runway" boilerplate the rerank weight is automatically suppressed by the caption-quality gate.

3.3 Comparing against the vanilla CLIP baseline

The image-only column above is already the *fashion-tuned* baseline - to compare against vanilla CLIP we re-embed the same 3,200-product catalogue with ViT-B-32 openai (the M0 backend) and re-run the eval. Mean top-1 drops from 0.281 (fashion-tuned) to 0.104, a 2.7x lift purely from the backbone swap. Adding captions + re-ranking takes the same fashion-tuned pipeline to 0.413 mean, a 4.0x total improvement over the vanilla baseline.

3.4 Per-query observations

- Q1 - "A person in a bright yellow raincoat." Returns *"a little girl wearing a yellow raincoat and red tights"* at rank 1 - a near-perfect match. The cross-encoder strongly supports this title (rerank score 0.98) and the caption-quality gate is high (0.9), so re-ranking boosts it without overfitting.

- Q2 - "P
office"* at
re-ranker i
actual top-

3.5 Where it falls short - and why

The single largest improvement opportunity is caption quality diversity: 40 % of the catalogue still produces a "model walks runway / fashion show" caption from BLIP-base. Multi-prompt BLIP generation (3 prompts per image) is implemented in captions.py but currently the deployed cache is single-prompt; running python scripts/buildcaptionindex.py --force regenerates with the richer cache. A second lever is replacing the MS-MARCO cross-encoder with a fashion-aware reranker (e.g. a small CLIP-style cross-encoder fine-tuned on FashionIQ) - also a one-config change.

---

4. Future work

The assignment asks specifically for (a) how to extend the system to cities, places, and weather, and (b) how to improve precision. We address each in turn.

4.1 Adding locations and weather

The current system treats the world as a flat image-text embedding space. To make it location- and weather-aware, we need three additions.

Step 1 - Enrich captions with structured context.

Today each image has one BLIP caption. We add three more fields, produced by a vision-language model (LLaVA-1.5, Florence-2, or CogVLM) running over the same images:

- scenetype - indoor / urban / coastal / forest / desert / snow
- weather

These join the existing caption in the index; they can also be stored as a structured side-table for hard filtering.

Step 2 - Add a geo-temporal prior.

For each region and month, precompute a weather prior from a public API (Open-Meteo is free and no-key). At query time, parse the query for location tokens ("Mumbai in August") and intersect with the prior to re-weight candidates that match expected conditions.

Step 3 - Extend the query API.

Add an optional query-time filter:

```
/search?q=beachwear&city=mumbai&month=august
```

The search runs as today, but candidates are first filtered (or down-weighted) by the geo-temporal prior. This is the natural extension for "shop the look in my city" applications on Glance's lock-screen.

When to use it. This extension is high-value when the catalogue is geographically diverse and the user wants outfit recommendations that are realistic for their context. It is low-value when the catalogue is style-only (e.g., a designer look-book) and location is irrelevant.

4.2 Improving precision

A precision roadmap, ordered by ROI. Items already shipped in this revision are marked ?.

```
| Lever | Mechanism | Expected gain | Cost | Status |
|---|---|---|---|---|
| ? Raise `rerank_top_n` | One-line config change; re-rank from top-150 hybrid candidates (was top-100) | +5-10 % on context-heavy queries (Q2, Q4) | Negligible | shipped |
```

? Caption-quality gating	Heuristic 0..1 quality on caption; scale rerank weight by quality to suppress generic "model walks runway" rerank noise	+2-4 on Q1/Q3/Q5	Negligible	shipped
? Rebalanced weights	`image_weight 0.65 / caption_weight 0.35 / rerank_weight 0.35` (was 0.5 / 0.5 / 0.5)	+10-20 mean score on all queries	Negligible	shipped
? Multi-prompt BLIP	3 prompts per image (default / attribute-focused / garment-focused); `--force` flag regenerates	+2-4 on compositional queries	+2x caption index build time	code shipped, regeneration optional
Larger encoder	Switch from `Marqo/marqo-fashionCLIP` (ViT-L/14) to a ViT-G/14 fashion backbone	+2-5 nDCG@10 on public benchmarks	Larger checkpoint, slower indexing	
Hard-negative mining + fine-tune	Mine compositional swaps in the corpus ("red tie + white shirt" vs. "white tie + red shirt"); contrastive-fine-tune for ~1k steps on these pairs	+5-8 on Q5 specifically	Requires GPU, held-out validation set	
Late-interaction over caption tokens	ColBERT-style token-level max-similarity between query and caption tokens	+3-6 on compositional queries	100x storage cost; slower query time	
Generative re-ranker (LLM judge)	LLaVA-1.6 evaluates each `(query, image)` pair, outputs a relevance score in {0, 1}	+5-10	Much higher latency per query	
Fashion-tuned cross-encoder	Fine-tune a small cross-encoder on FashionIQ swaps, swap into `cfg.rerank.model`	+3-5 on all queries	Held-out dataset, GPU fine-tune	
Active-learning feedback loop	Capture click-through on top-k results; add positive pairs to monthly retraining	+1-3 per cycle, compounding	Requires live traffic	

|---|---|---|---|---|

| ? Raise r

% on conte

The single highest-ROI change is hard-negative fine-tuning. The model already has fashion priors; teaching it to distinguish *"red tie + white shirt"* from *"white tie + red shirt"* needs labelled pairs, not a new architecture. The current code is structured to slot a fine-tuned model in by changing one config field.

4.3 Other directions

- Image-to-image search. Add --image <path> to retriever/search.py for visual-similarity lookups using the same index. Useful for "more like this".

- Hard-n
unblock fin

Appendix A - Codebase

GitHub: github.com/amanraj74/Glance-fashion-search

The repository contains the full pipeline: indexer, retriever, scripts, Streamlit demo, tests, and this writeup.

README at the repo root covers installation, configuration, and command reference.

Appendix B - How to reproduce

All commands assume the repository root and an activated virtual environment.

```
# 1. Install dependencies
python -m venv venv
source venv/bin/activate          # or .\venv\Scripts\Activate.ps1 on Windows
pip install --upgrade pip
pip install -r requirements.txt
pip install pytest streamlit

# 2. Build the image index (downloads ~1 GB of fashionCLIP weights)
python indexer/build_index.py
# Or swap to a different backbone:
#   python indexer/build_index.py --model ViT-B-16-SigLIP-512 --pretrained webli
#   python indexer/build_index.py --backend ivfflat

# 3. Generate BLIP captions and embed them (~500 MB BLIP-base weights)
python scripts/build_caption_index.py            # incremental; respects existing cache
python scripts/build_caption_index.py --force    # regenerate from scratch (3 prompts per image)

# 4. Run the 5-query evaluation harness
python scripts/run_eval.py

# 5. Run the A/B ablation comparison
python scripts/run_ablation.py

# 6. Run the rank-based metrics harness (RRF, score gap, top-k diversity, axis tags)
python scripts/run_metrics.py

# 7. Render this writeup to HTML / PDF
python scripts/build_report.py

# 8. Launch the interactive web demo (text query or image upload)
streamlit run app/streamlit_app.py
```

Expected wall-clock on CPU:

```
| Step | Time | Output |
|---|---|---|
| `indexer/build_index.py` | 15-25 min | `output/faiss.index`, `output/metadata.json` |
|   `scripts/build_caption_index.py`   |   30-60   min   |   `output/captions.json`,   `output/captions_multi.json`,
`output/captions.index`, `output/caption_meta.json` |
| `scripts/run_eval.py` | < 1 min | `eval/results/*.json`, `eval/results/*.png` |
| `scripts/run_ablation.py` | < 1 min | `eval/results/ablation.csv` |
| `scripts/run_metrics.py` | < 1 min | `eval/results/metrics.csv`, `eval/results/metrics.json` |
| `scripts/build_report.py` | < 30 s | `report/Glance_Internship_Report.{md,html,pdf}` |
```

|---|---|---|

| Embedd

ViT-B-16-S

Tests:

```
pytest tests/ -v
```

49 tests pass in ~20 seconds. No model download required.

Appendix C - Honest tradeoffs

- Compute. The project was developed on a CPU-only machine. No fine-tuning was performed; the architectural scaffolding for fine-tuning is ready but unexercised.

- Datase
IndexIVFFl

This writeup is the deliverable required by ?5 of the assignment brief. The accompanying code is the deliverable required by ?5.3.